Artificial Intelligence
人工智能实验

搜索算法

中山大学计算机学院
2026年春季

目录

1. 理论课内容回顾

1.1 盲目搜索

1.2 启发式搜索

1.3 博弈树搜索

1.4 高级搜索

2. 实验任务

2.1 盲目搜索解决迷宫问题（无需提交）

2.2 启发式搜索解决15-Puzzle问题

2.3 博弈树搜索实现象棋AI （无需提交）

2.4 遗传算法求解 TSP 问题

3. 作业提交说明

目录

1. 理论课内容回顾

1.1 盲目搜索

1.2 启发式搜索

1.3 博弈树搜索

1.4 高级搜索

2. 实验任务

2.1 盲目搜索解决迷宫问题（无需提交）

2.2 启发式搜索解决15-Puzzle问题

2.3 博弈树搜索实现象棋AI （无需提交）

2.4 遗传算法求解 TSP 问题

3. 作业提交说明

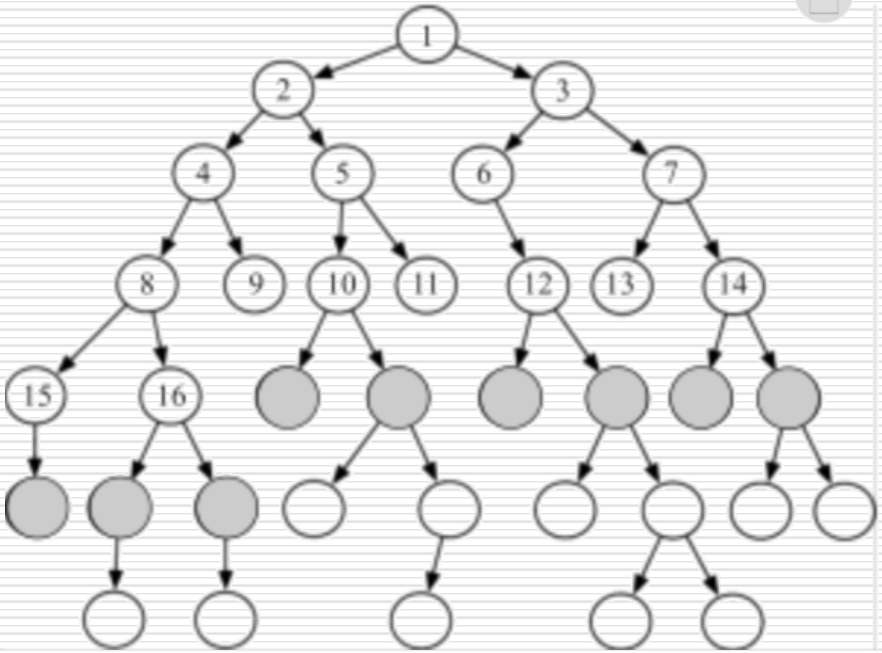
1.1 盲目搜索

1. 宽度优先搜索
2. 深度优先搜索
3. 深度受限搜索
4. 迭代加深搜索
5. 双向搜索

1.1 盲目搜索

1. 宽度优先搜索（BFS）

- **解的深度为d、最大分支为b;**



算法 宽度优先搜索 BFS

输入: 初始状态 s_0, 动作集合, 转移模型 T, 目标检测函数

```
初始化队列 queue ← [s0]
while queue 非空 do
    s ← queue.pop()
    for s 的所有可行动作 a do
        获取下一状态 s' = T(s, a)
        if s' 是目标 then
            return 搜索路径
        else
            queue.append(s')
        end if
    end for
end while
return 问题无解
```

- **节点扩展顺序与目标节点的位置无关;**
- **用一个先进先出（FIFO）队列实现。**

- **完备性;**
- **非最优;**
- **时间复杂度O(b^d);**
- **空间复杂度O(b^d);**

1.1 盲目搜索

2. 深度优先搜索（DFS）

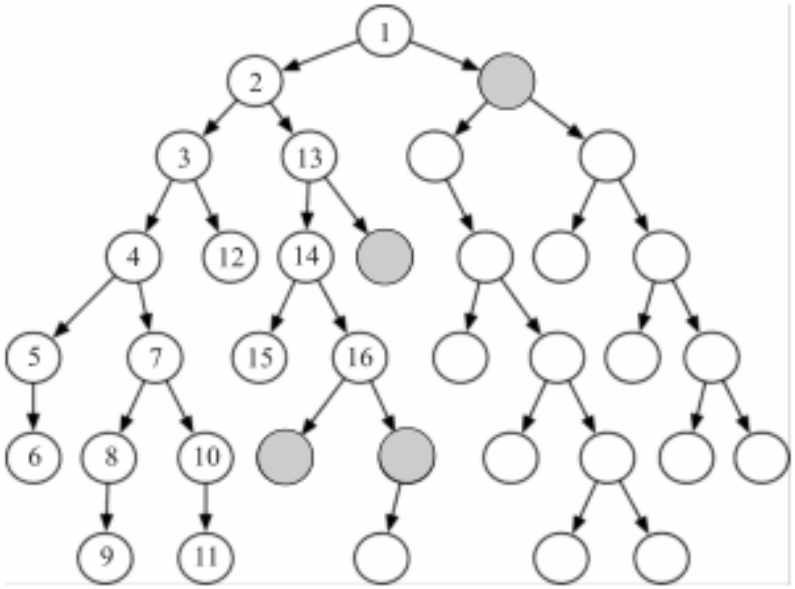


图 3-5　深度优先搜索中节点的扩展顺序

云栖社区 yq.aliyun.com

算法 深度优先搜索 DFS

输入: 初始状态 s_0, 动作集合, 转移模型 T, 目标检测函数

```
if s0 下无可执行动作 then
    return 当前无解
else if s0 是目标 then
    return 搜索路径
else
    for all s0 的可行动作 a do
        获取下一状态 s' = T(s0, a)
        若 s' 未访问, 则以 s' 为初始状态递归执行 DFS
    end for
end if
```

- **节点扩展顺序与目标节点的位置无关；**
- **用一个后进先出堆栈实现。**

1.1 盲目搜索

3. 深度受限搜索（Deep-limited Search）

算法 有界深度搜索 DLS

输入: 初始状态 s_0, 动作集合, 转移模型 T, 目标检测函数, 深度限制 d

```
if d = 0 then
    return 当前无解
else if s0 是目标 then
    return 搜索路径
else
    for all s0 的可行动作 a do
        获取下一状态 s' = T(s0, a)
        若 s' 未访问, 则以 s' 为初始状态, d − 1 为深度限制, 递归执行 DLS
    end for
end if
```

- **若状态空间无限，深度优先搜索可能会出现循环，搜索失败；**
- **通过预定一个深度限制L 来解决这个问题。**

1.1 盲目搜索

4. 迭代加深搜索（Iterative Deepening Search）

算法 迭代加深搜索 IDS

输入: 初始状态 s_0, 动作集合, 转移模型 T, 目标检测函数

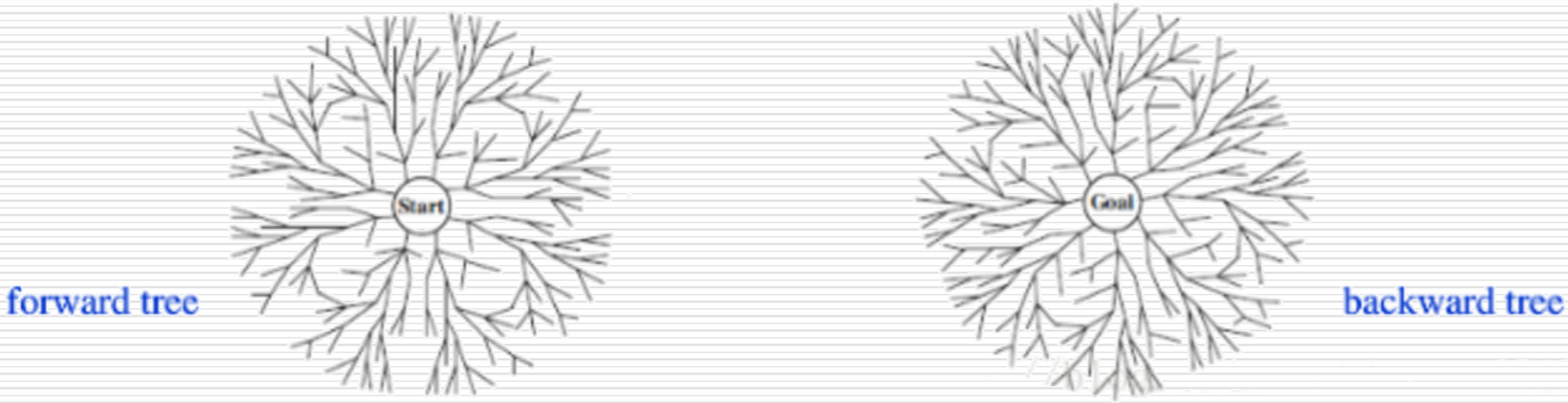
```
for d = 1 to +∞ do
    以 s0 为初始状态, d 为深度限制执行 DLS
    if 本次搜索找到解 then
        return 搜索路径
    else if 本次搜索所有节点深度小于 d then
        return 搜索失败
    end if
end for
```

- **深度受限搜索的L值难以选择；**
- **迭代加深搜索：以深度优先搜索相同的顺序访问搜索树的节点，但先访问节点的累积顺序实际是宽度优先。**

1.1 盲目搜索

5. 双向搜索（Bidirectional search）

它同时进行两个搜索：一个是从初始状态向前搜索，二另一个则从目标向后搜索。当两者在中间相遇时停止。



- **同时进行两个搜索：一个是从初始状态向前搜索，二另一个则从目标向后搜索；**
- **当两者在中间相遇时停止；**
- **在一定程度上能减小复杂度。**

1.1 盲目搜索

盲目搜索策略评估

Evaluation of Uninformed Tree-search Strategies

无信息树搜索策略评价

Criterion	Breadth First	Uniform Cost	Depth First	Depth Limited	Iterative Deepening	Bidirectional
Complete	Yes^a	$\text{Yes}^{a,b}$	No	No	Yes^a	$\text{Yes}^{a,d}$
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$
Optimal	Yes^c	Yes	No	No	Yes^c	$\text{Yes}^{c,d}$

Where

- b -- maximum branching factor of the tree
- d -- depth of the shallowest solution
- m -- maximum depth of the tree
- l -- the depth limit

- a -- complete if b is finite
- b -- complete if step costs ϵ for positive
- c -- optimal if step costs are all identical
- d -- if both directions use breadth-first search

http://blog.csdn.net/c11556913

1.2 启发式搜索

1. 基础概念
2. A*
3. IDA*
4. 启发式函数的设计

1.2 启发式搜索

1. 基础概念

- 启发式搜索又叫有信息的搜索，利用问题所拥有的启发信息来引导搜索，达到减少搜索范围，降低问题复杂度的目的。
  - 无信息搜索对所有的可能路径节点一视同仁，而启发式搜索可以指导搜索向最有希望的方向前进
  - 如何评估一个节点的重要性？ - 评估函数

$$f(x) = h(x) + g(x)$$

  - 其中g(x)是从初始节点到节点x付出的实际代价；而h(x)是从节点x到目标节点的最优路径的估计代价。h(x)建模了启发式搜索问题中的启发信息，是算法的关键。启发式函数的设计非常重要，合理的定义才能让搜索算法找到一个最优的问题解。

1.2 启发式搜索

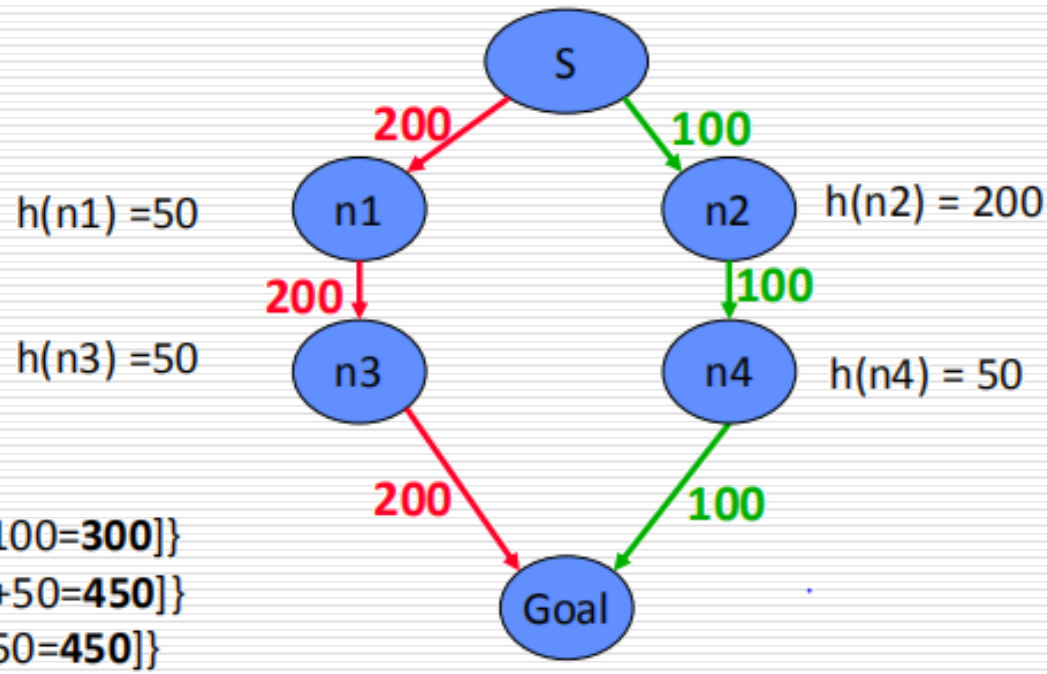
2. A*搜索算法

- A*算法可以看作是BFS算法的升级版，在原有的BFS算法的基础上加入了启发式信息。
  - 算法描述：从起始节点开始，不断查询周围可到达节点的状态并计算它们的f(x)，h(x)与g(x)的值，选取评估函数f(x)最小的节点进行下一步扩展，并同时更新已经被访问过的节点的g(x)，直到找到目标节点；
  - 算法优缺点：拥有BFS速度较快的优点，但是因为它要维护“开启列表”以及“关闭列表”，并且需要反复查询状态。因此它的空间复杂度是指数级的。

2. A*搜索算法

→ step cost = 200

→ step cost = 100



g(n)+h(n)=f(n)

{S} → {n1 [200+50=**250**], n2 [200+100=**300**]}

→ {n2 [100+200=**300**], n3 [400+50=**450**]}

→ {n4 [200+50=**250**], n3 [400+50=**450**]}

→ {goal [300+0=**300**], n3 [400+50=**450**]}

1.2 启发式搜索

3. IDA*（迭代加深A*）搜索算法

- IDA* 是迭代加深深度优先搜索算法（IDS）的扩展。因为它不需要去维护表，因此它的空间复杂度远远小于A*。在搜索图为稀疏有向图的时候，它的性能会比A*更好。
  - 算法描述：在算法迭代的每一步，IDA*都进行深度优先搜索，在某一步所有可访问节点对应的最小可估价函数值大于某个给定的阈值的时候，将会剪枝；
  - 算法优缺点：当问题要求空间复杂度比较低的时候，IDA*更有优势。

1.2 启发式搜索

4. 启发式函数设计

- 启发式函数h(n)告诉算法从任何节点到目标节点的最小代价估计值，其选取很大程度影响算法性能。

h(n) 的值	描述	性能变化
h(n) = 0	只有g(n)起作用，退化为Dijkstra算法	保证找到最短路径
h(n) <= h*(n)		保证能找到最短路径
h(n) = h*(n)	只遵循最佳路径不会扩展其它节点	运行速度快并且能找到最短路径
h(n) > h*(n)		不能保证找到最短路径

1.2 启发式搜索

4. 启发式函数设计

- 启发式函数h(n)告诉算法从任何节点到目标节点的最小代价估计值，其选取很大程度影响算法性能。
- 性质1：可采纳的（admissible）
  - 当评估函数的预估值小于等于真实值时，算法必然可以找到一条从起始节点到最终节点的最短路径。这种性质叫做相容。

$$h(n) \le h^*(n)$$

- 性质2： 单调的 （consistent）
  - 当节点n的评估函数值永远小于等于它的扩展节点n’的评估函数值加上扩展代价时，则启发式函数设计是单调的。

$$h(n) \le \cos t(n, n') + h(n')$$

1.2 启发式搜索

4. 启发式函数设计

- 不同的应用场景下有很多可选择的启发式函数。比如在网格地图中，一般使用以下几种启发式函数：
  - 在正方形网格中，允许向4邻域的移动，使用曼哈顿距离（L1）
  - 在正方形网格中，允许向8邻域的移动，使用对角线距离（L∞）等等
- 启发函数没有限制，大家可以多尝试几种。

1.3 博弈树搜索

1. 两玩家零和博弈问题
2. 博弈树
3. Minimax搜索
4. Alpha-beta剪枝

1.3 博弈树搜索

1. 两玩家零和博弈问题

- 两名玩家轮流行动，进行博弈
  - 有限性：行动的个数有限
  - 确定性：不存在随机性
  - 信息完备性：博弈双方知道所处状态的全部信息
  - 零和性：一方的损失相当于另一方的收益，总收益为0
    - 结局有三种可能：玩家A获胜、玩家B获胜、平局（或两种可能，无平局）

1.3 博弈树搜索

1. 两玩家零和博弈问题

- **An example: Rock, Paper, Scissors**
  - Scissors cut paper, paper covers rock, rock smashes scissors
  - Represented as a matrix: Player I chooses a row, Player II chooses a column
  - Payoff to each player in each cell (Pl.I / Pl.II)
  - 1: win, 0: tie, -1: loss
  - so it's zero-sum

		Player II		
		R	P	S
Player I	R	0/0	-1/1	1/-1
	P	1/-1	0/0	-1/1
	S	-1/1	1/-1	0/0

1.3 博弈树搜索

2. 博弈树

- 节点（node）：表示问题的状态（state）。
  - 分为内部节点（interior node）和叶子节点（leaf node）
- 扩展节点：行动（action）。
- 双方轮流扩展节点：两个玩家的行动逐层交替出现。
- 博弈树的值：博弈树搜索的目的，找出对双方都是最优的子节点的值。给定叶子节点的效益值，搜索内部节点的值。
- 评价函数：对当前节点的优劣评分。在有深度限制时，原来的内部节点会充当叶子节点的作用，此时以评价函数值作为效益值的估计。

1.3 博弈树搜索

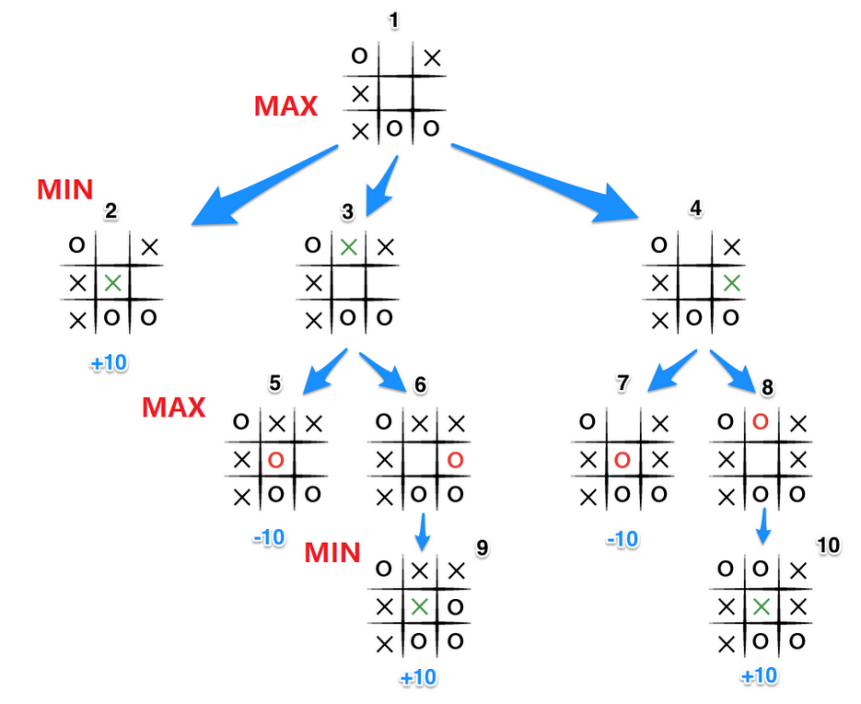
2. 博弈树

- 要提高博弈问题求解程序的效率，应作到如下两点：
  - 改进生成过程，使之只生成好的走步，如按棋谱的方法生成下一步；
  - 改进测试过程，使最好的步骤能够及时被确认。
- 要达到上述目的有效途径是使用启发式方法引导搜索过程，使其只生成可能赢的走步。而这样的博弈程序应具备：
  - 一个好的（即只产生可能赢棋步骤的）生成过程。
  - 一个好的静态估计函数。
- 下面介绍博弈中两种最基本的搜索方法。

1.3 博弈树搜索

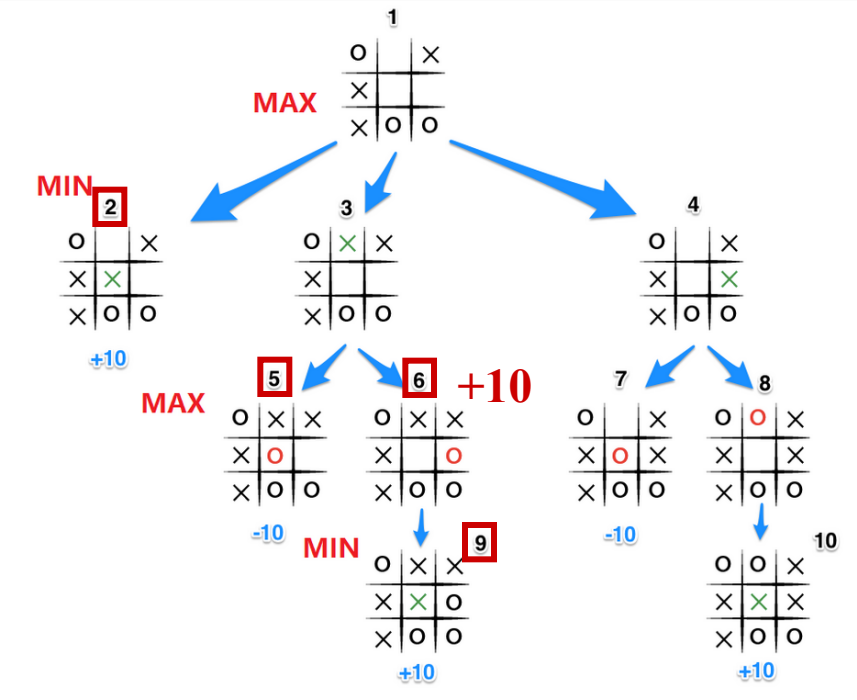
3. Minimax搜索

- 假设：
  - 玩家A和玩家B的行动逐层交替；
  - A和B的利益关系对立，即假设A要使分数更大，B就要使分数更小；
  - A和B均采取最优策略。
- Minimax搜索：找到博弈树中内部节点的值，其中Max节点（A）的每一步扩展要使收益最大，Min节点（B）的扩展要使收益最小。



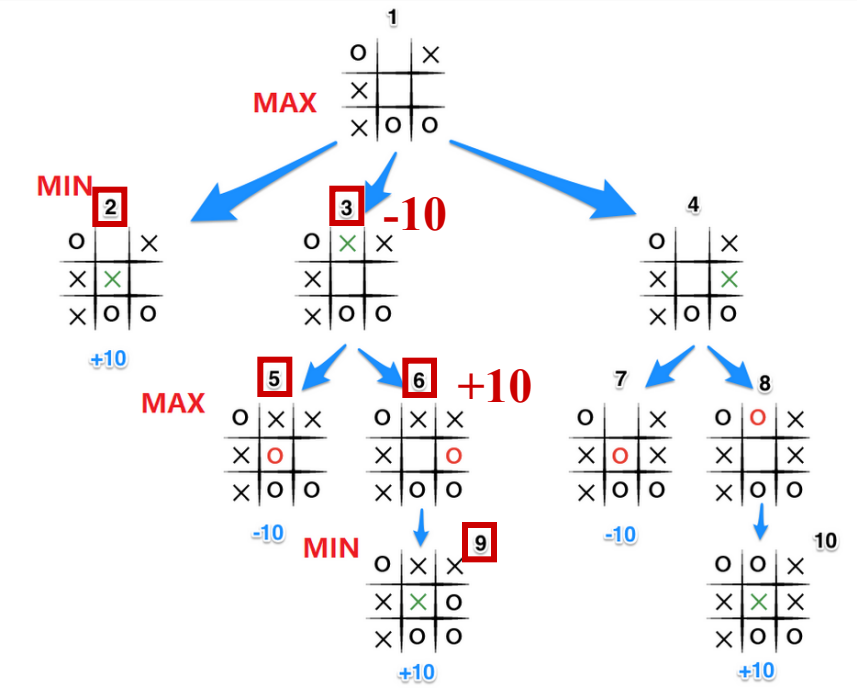
1.3 博弈树搜索

3. Minimax搜索



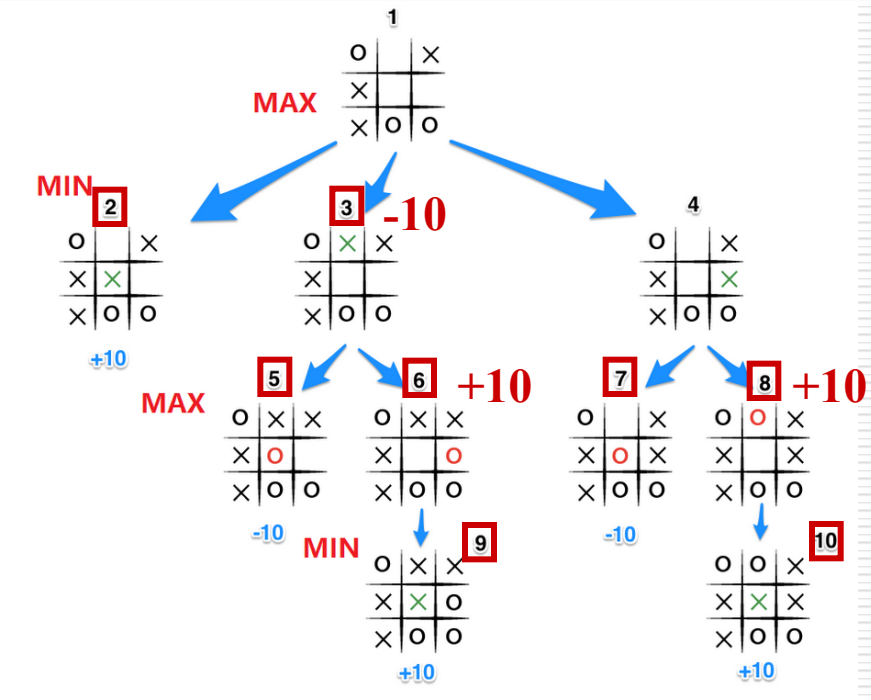
1.3 博弈树搜索

3. Minimax搜索



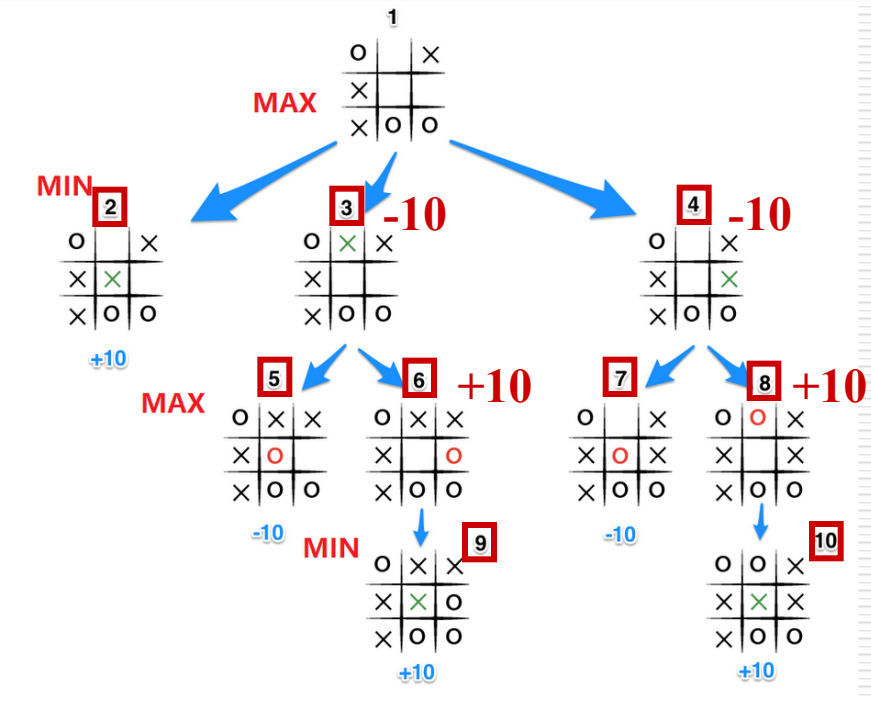
1.3 博弈树搜索

3. Minimax搜索



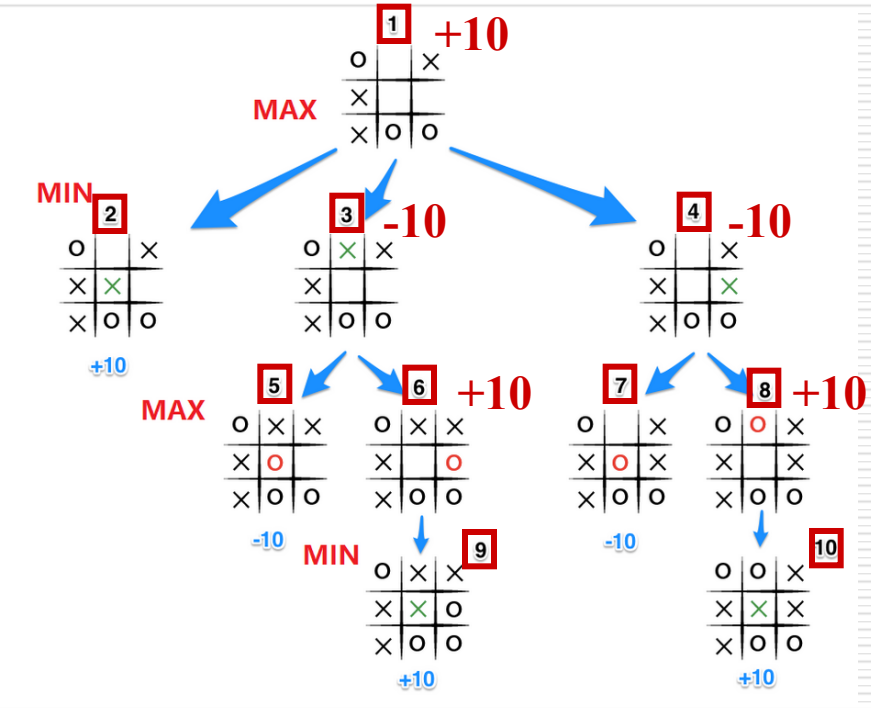
1.3 博弈树搜索

3. Minimax搜索



1.3 博弈树搜索

3. Minimax搜索



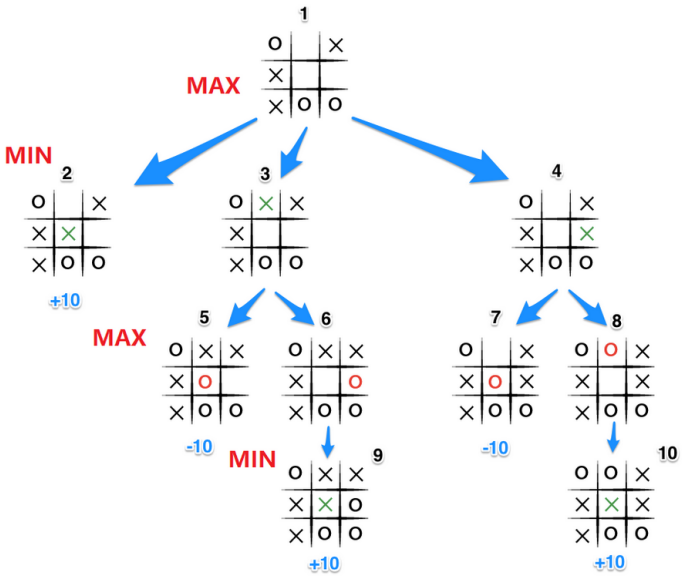
1.3 博弈树搜索

3. Minimax搜索

```
function MINIMAX-DECISION(state) returns an action
  return arg max_{a ∈ ACTIONS(s)} MIN-VALUE(RESULT(state, a))
```

```
function MAX-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← −∞
  for each a in ACTIONS(state) do
    v ← MAX(v, MIN-VALUE(RESULT(s, a)))
  return v
```

```
function MIN-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← ∞
  for each a in ACTIONS(state) do
    v ← MIN(v, MAX-VALUE(RESULT(s, a)))
  return v
```



1.3 博弈树搜索

4. Alpha-beta剪枝

- Minimax搜索：随着博弈的进行，必须检查的游戏状态的数目呈指数增长；
  - **我们只需要知道博弈过程所对应路径上的节点值；**
- Alpha-beta剪枝：剪掉不可能影响决策的分支，尽可能地消除部分搜索树。
  - **Max节点记录alpha值，Min节点记录beta值**
  - **Max节点的alpha剪枝：效益值 ≥ 任何祖先Min节点的beta值**
  - **Min节点的beta剪枝：效益值 ≤ 任何祖先Max节点的alpha值**

1.3 博弈树搜索

4. Alpha-beta剪枝

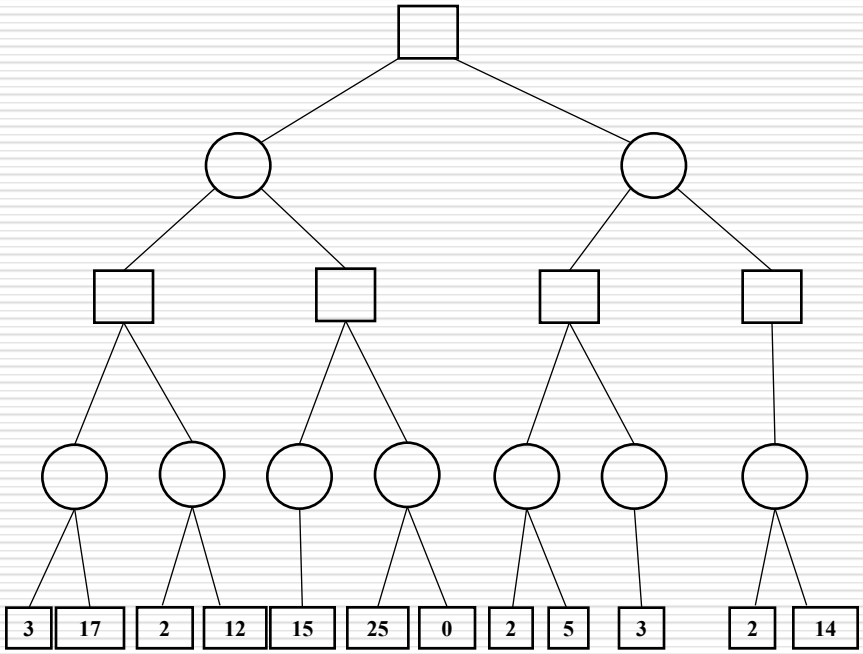
```
function ALPHA-BETA-SEARCH(state) returns an action
  v ← MAX-VALUE(state, −∞, +∞)
  return the action in ACTIONS(state) with value v
```

```
function MAX-VALUE(state, α, β) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← −∞
  for each a in ACTIONS(state) do
    v ← MAX(v, MIN-VALUE(RESULT(s,a), α, β))
    if v ≥ β then return v
    α ← MAX(α, v)
  return v
```

```
function MIN-VALUE(state, α, β) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← +∞
  for each a in ACTIONS(state) do
    v ← MIN(v, MAX-VALUE(RESULT(s,a) , α, β))
    if v ≤ α then return v
    β ← MIN(β, v)
  return v
```

1.3 博弈树搜索

4. Alpha-beta剪枝

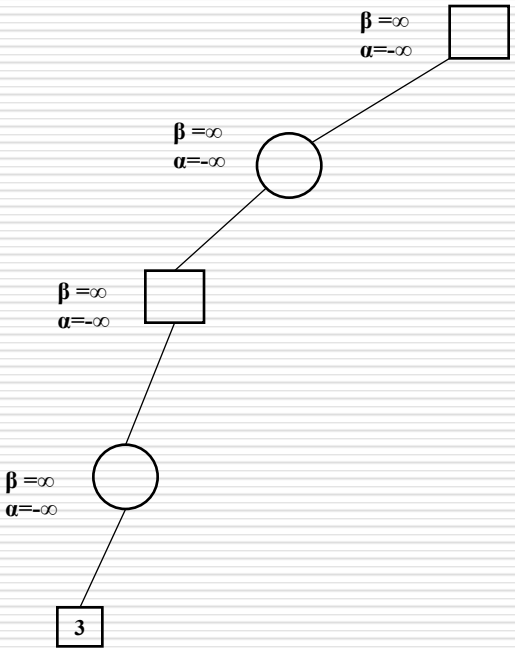


方形：Max节点
圆形：Min节点

请写出对该博弈树进行搜索的过程，要求使用结合Alpha-beta剪枝的深度优先Minimax算法。

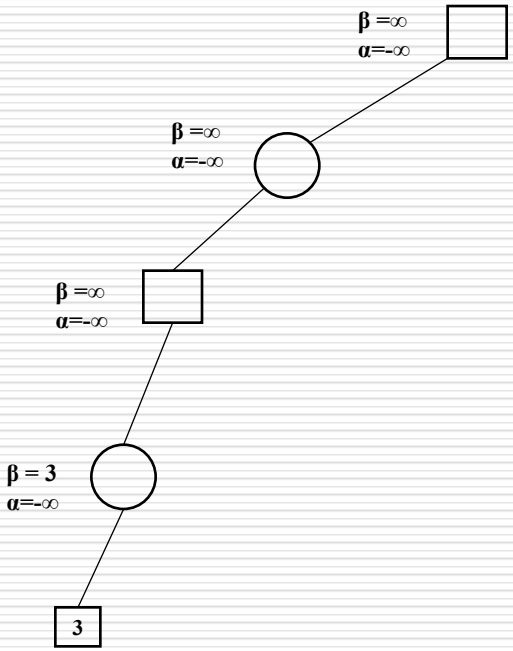
1.3 博弈树搜索

4. Alpha-beta剪枝



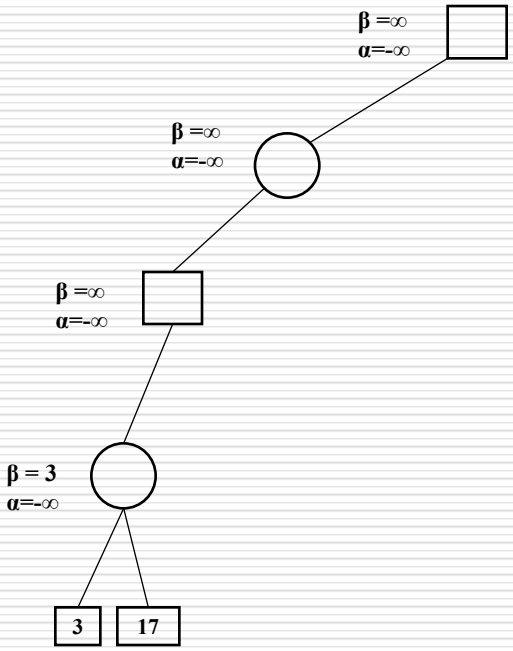
1.3 博弈树搜索

4. Alpha-beta剪枝



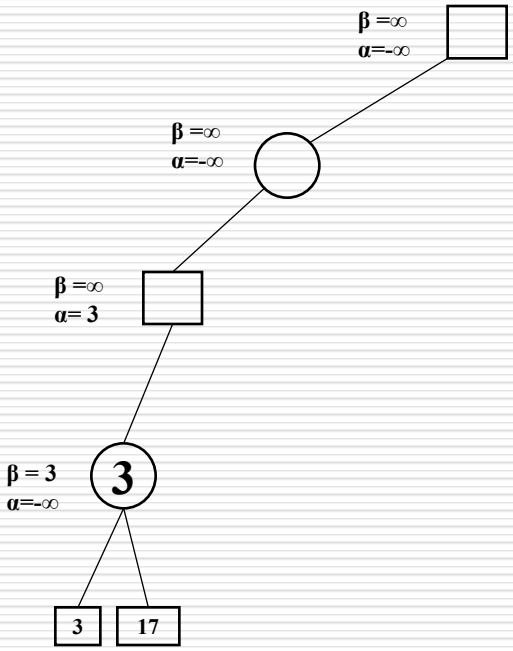
1.3 博弈树搜索

4. Alpha-beta剪枝



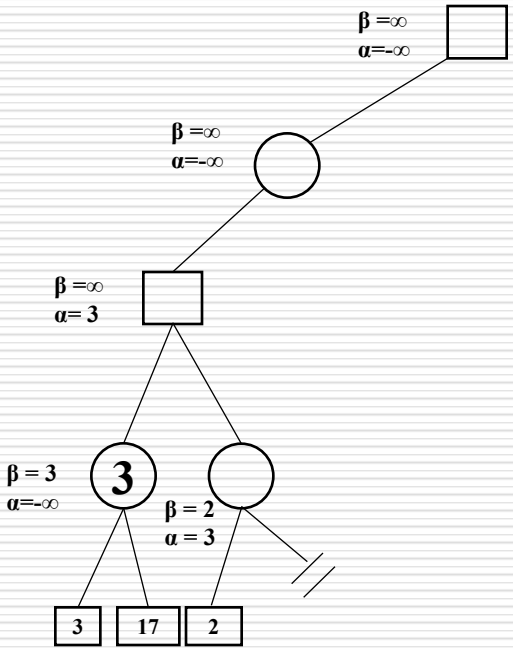
1.3 博弈树搜索

4. Alpha-beta剪枝



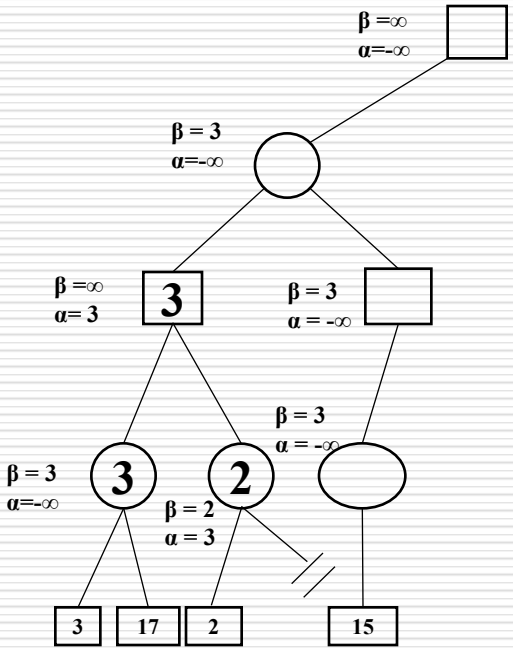
1.3 博弈树搜索

4. Alpha-beta剪枝



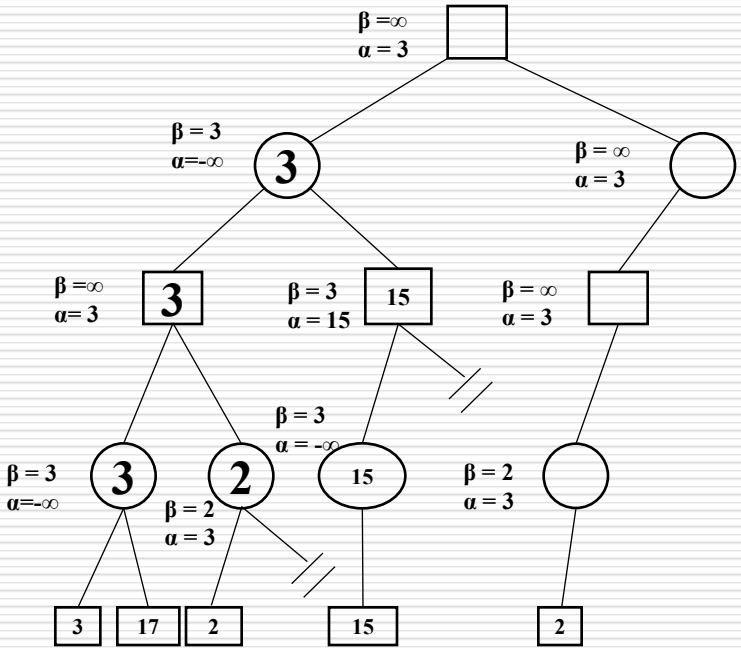
1.3 博弈树搜索

4. Alpha-beta剪枝



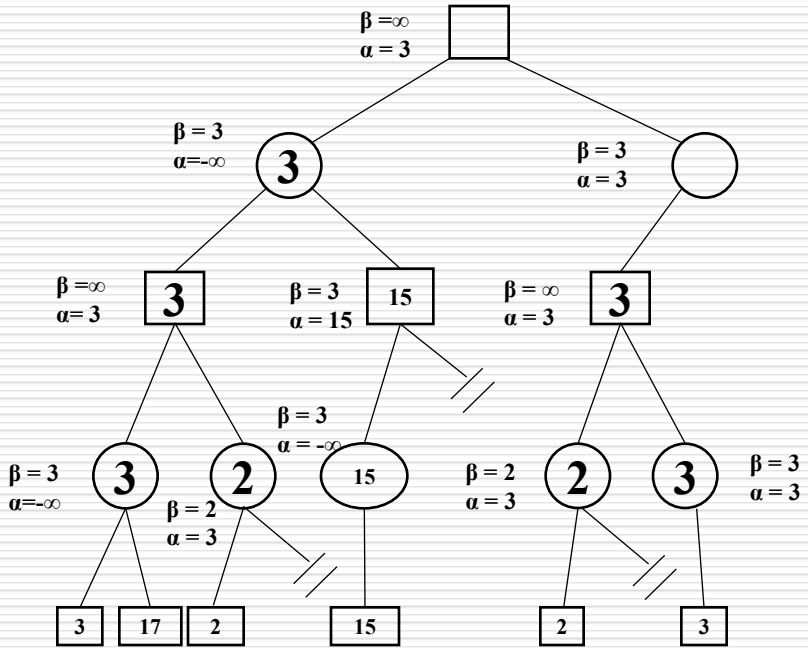
1.3 博弈树搜索

4. Alpha-beta剪枝



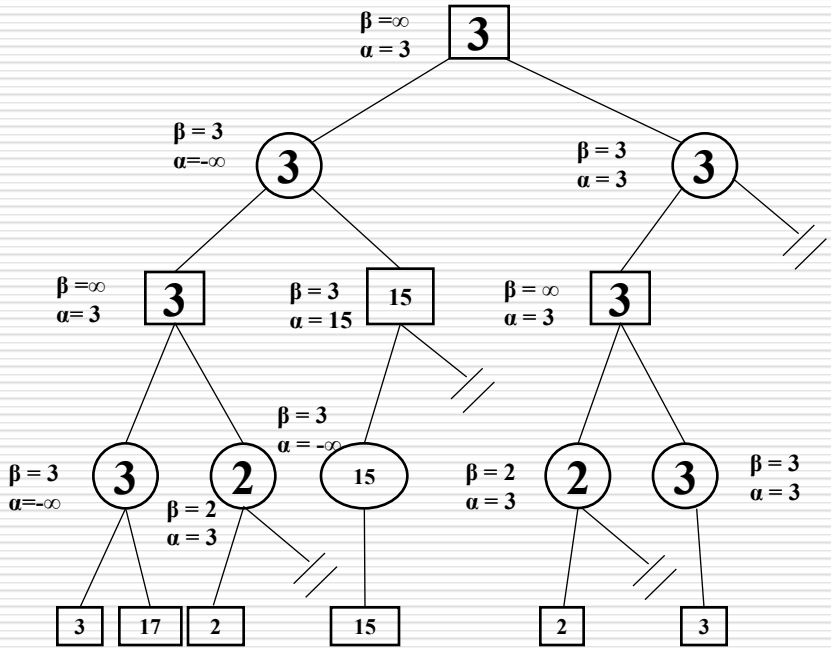
1.3 博弈树搜索

4. Alpha-beta剪枝



1.3 博弈树搜索

4. Alpha-beta剪枝



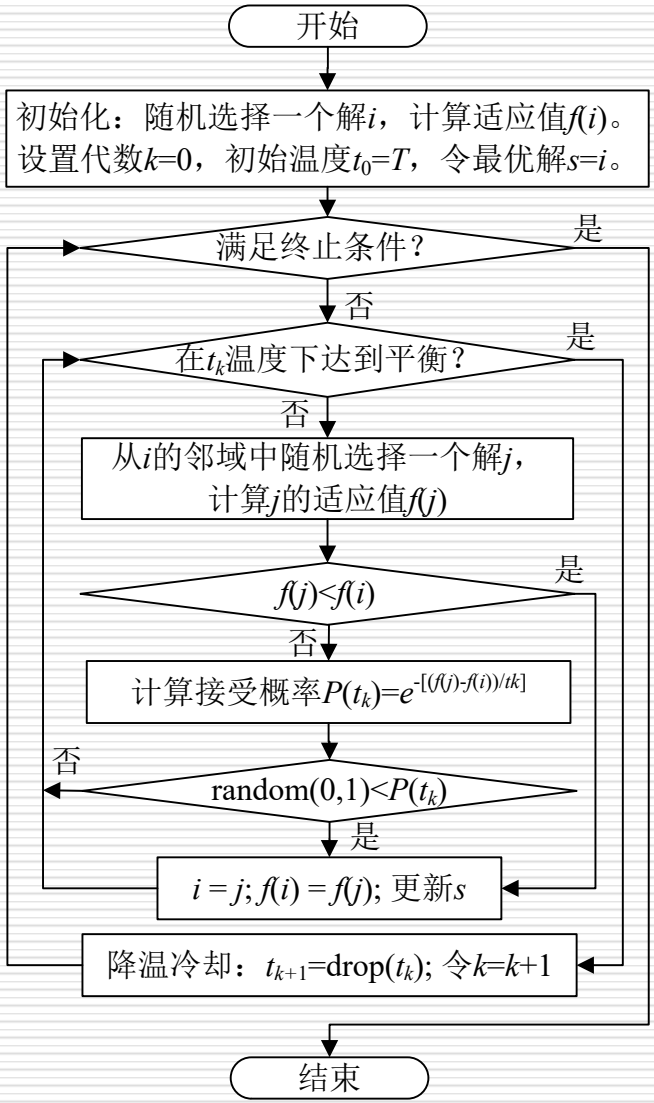
1.4 高级搜索

1. 模拟退火算法

2. 遗传算法

3. 旅行商问题

1. 模拟退火算法



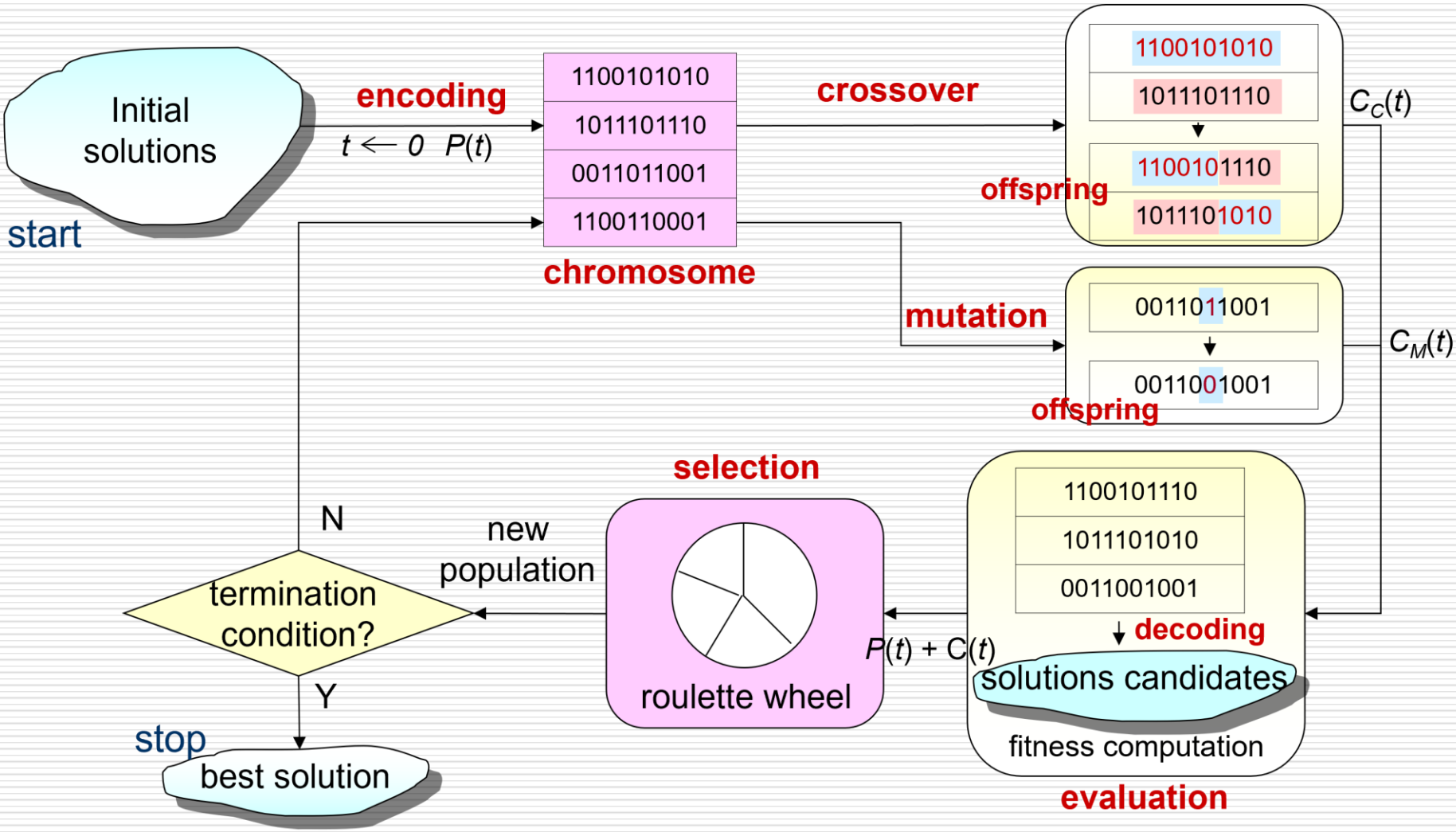

```
//功能：模拟退火算法伪代码
//说明：本例以求问题最小值为目标
//参数：T为初始温度; L为内层循环次数

procedure SA
  //Initialization
    Randomly generate a solution X0, and calculate its
    fitness value f(X0);
  Xbest=X0; k=0; tk=T;
  while not stop
    //The search loop under the temperature tk
    for i=1 to L  //The loop times
      Generate a new solution Xnew based on the current
      solution Xk, and calculate its fitness value f(Xnew).
      if f(Xnew) < f(Xk)
          Xk = Xnew;
          if f(Xk) < f(Xbest) Xbest=Xk;
          continues;
      end if
      Calculate P(tk)=e^-[(f(Xnew)-f(Xk))/tk];
       if random(0,1) < P
          Xk = Xnew;
      end if
    end for
    //Drop down the temperature
    tk+1=drop(tk); k=k+1;
  end while
  print Xbest
end procedure
```

1.4 高级搜索

1. 模拟退火算法

功能意义	基本要素	设置方法
影响模拟退火算法全局搜索性能的重要因素之一。 实验表明，初温越大，获得高质量解的几率越大，但花费的计算时间将增加。	初始温度	1、均匀抽样一组状态，以各状态目标值的方差定初温 2、随机产生一组状态，以两两状态间最大差值定初温 3、利用经验公式给出初温
状态空间与状态产生函数。 邻域函数（状态产生函数）应尽可能保证产生的候选解遍布全部解空间。	邻域函数	候选解一般采用按照某一概率密度函数对解空间进行随机采样来获得。 概率分布可以是均匀分布、正态分布、指数分布等等
指从一个状态X_k(一个可行解)向另一个状态X_{new}(另一个可行解)的转移概率，通俗的理解是接受一个新解为当前解的概率	接受概率	一般采用Metropolis准则 $P_{ij}^T=\begin{cases}1, & \text{if } E(j)\le E(i)\\ e^{-(\frac{E(j)-E(i)}{KT})}=e^{-(\frac{\Delta E}{KT})}, & \text{otherwise}\end{cases}$
指从某一较高温状态t_0向较低温状态冷却时的降温管理表，或者说降温方式	冷却控制	1、经典模拟退火算法的降温方式 $t_k=\frac{t_0}{\lg(1+k)}$ 2、快速模拟退火算法的降温方式 $t_k=\frac{t_0}{1+k}$
内层平衡也称Metropolis抽样稳定准则，用于决定在各温度下产生候选解的数目	内层平衡	1、检验目标函数的均值是否稳定 2、连续若干步的目标值变化较小 3、预先设定的抽样数目，内循环代数
算法的终止条件	终止条件	1、设置终止温度的阈值 2、设置外循环迭代次数 3、算法搜索到的最优值连续若干步保持不变 4、检验系统熵是否稳定

2. 遗传算法


Initial solutions
start
encoding
$t \leftarrow 0$ $P(t)$
1100101010
1011101110
0011011001
1100110001
chromosome
crossover
1100101010
1011101110
1100101110
1011101010
offspring
$C_C(t)$
mutation
0011011001
0011001001
offspring
$C_M(t)$
selection
roulette wheel
new population
N
termination condition?
Y
stop
best solution
$P(t) + C(t)$
1100101110
1011101010
0011001001
decoding
solutions candidates
fitness computation
evaluation

2. 遗传算法

算法 遗传算法解决 TSP

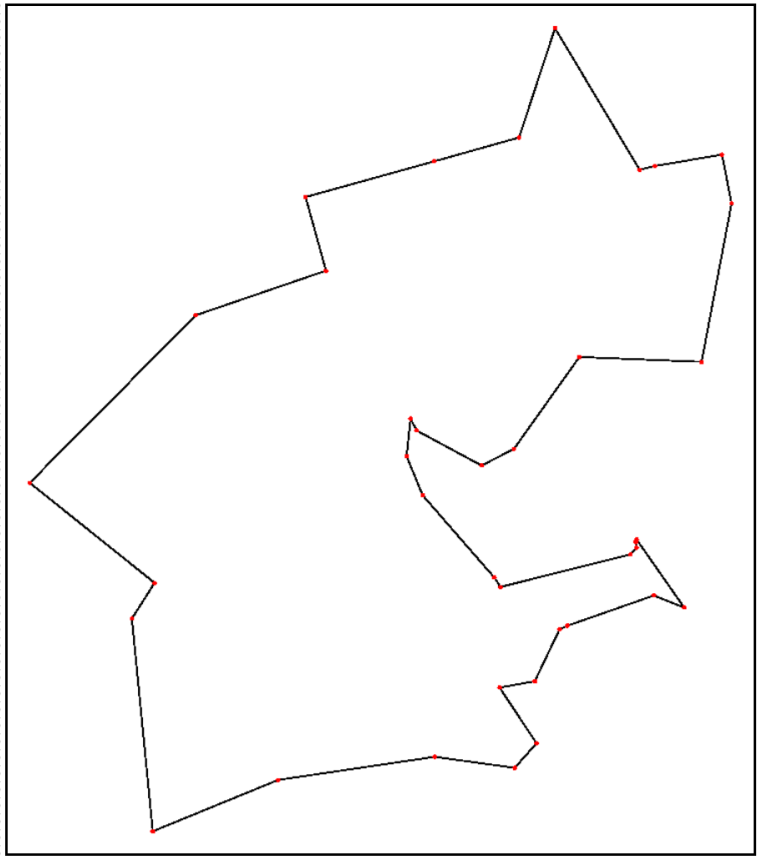
输入: n 个城市的坐标

```
随机生成 10 个不同个体的初始种群 P(0)
for t = 0 to 最大迭代轮数 do
    初始化后代种群 C(t)
    for i = 1 to 每一代后代数目 do
        依据适应度, 从 P(t) 中选取一对父母 p1, p2
        利用交叉操作, 从 p1, p2 产生后代 c1, c2
        以一定概率对 c1, c2 进行变异操作
        C(t) ← {c1, c2}
    end for
    依据适应度, 从 P(t) ∪ C(t) 选取优异个体作为 P(t + 1)
end for
依据适应度, 从最后一代种群中选择最优个体作为问题的解
```

1.4 高级搜索

3. 旅行商问题

- 旅行商问题(TSP)是最为广泛研究的组合优化问题之一.
- 问题描述: 一个旅行商寻找环游n个城市(不重复)的最短路径.



3. 旅行商问题——解的表示

- **表示方法一：随机排列**
  - 对于n个城市的TSP问题，解表示为1-n的排列

染色体表示

5	4	6	9	2	1	7	8	3

环游顺序: 5-4-6-9-2-1-7-8-3-5

- **表示方法二：随机数排序**
  - 生成n个(0,1)范围内的随机数
  - 这n个随机数的排序对应旅游城市的顺序

染色体表示

1	2	3	4	5	6	7	8	9
0.23	0.82	0.45	0.74	0.87	0.11	0.56	0.69	0.78

环游顺序: 6-1-3-7-8-2-9-4-5-6

3. 旅行商问题——部分映射交叉

算法 染色体交叉操作

输入: 染色体 v_1, v_2 及其长度 l

1: $R \leftarrow \varnothing$
   # 切割染色体
2: 随机生成两个下标 s, t, 满足 $0 < s < t < l-1$
   # 交叉染色体
3: $v_1' \leftarrow [v_1[0], \cdots, v_1[s-1], v_2[s], \cdots, v_2[t], v_1[t+1], \cdots, v_1[l-1]]$
4: $v_2' \leftarrow [v_2[0], \cdots, v_2[s-1], v_1[s], \cdots, v_1[t], v_2[t+1], \cdots, v_2[l-1]]$
   # 基因映射
5: 依据 $[v_1[s], \cdots, v_1[t]]$ 与 $[v_2[s], \cdots, v_2[t]]$ 建立映射关系
   # 生成后代染色体
6: 依据映射关系修改 v_1', v_2' 非交叉部分

step 1 : 随机选择下标 s,t

			s			t			
parent 1	1	2	3	4	5	6	7	8	9
parent 2	5	4	6	9	2	1	7	8	3

step 2: 交叉子串

proto-child 1	1	2	6	9	2	1	7	8	9
proto-child 2	5	4	3	4	5	6	7	8	3

step 3 : 确定映射关系

6	9	2	1
3	4	5	6

$1 \leftrightarrow 6 \leftrightarrow 3$
$2 \leftrightarrow 5$
$9 \leftrightarrow 4$

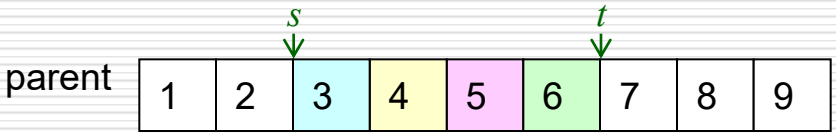
step 4 : 生成后代染色体

offspring 1	3	5	6	9	2	1	7	8	4
offspring 2	2	9	3	4	5	6	7	8	1

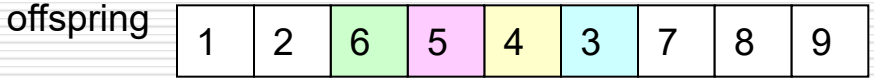
其他交叉操作: order crossover (OX), cycle crossover (CX), position-based crossover, order-based crossover等. 感兴趣的同学可自行尝试.

3. 旅行商问题——倒置变异

step 1: 随机选择下标 s,t



step 2 : 将s-t中间部分倒置



算法 染色体变异操作

输入: 染色体 v 及其长度 l

1: $R \leftarrow \varnothing$

切割染色体

2: 随机生成两个下标 s, t, 满足 $0 < s < t < l-1$

倒置

3: $v' \leftarrow [v[0], \cdots, v[s-1], v[t], v[t-1], \cdots, v[s+1], v[s], v[t+1], \cdots, v[l-1]]$

4: **return** v'

目录

1. 理论课内容回顾

1.1 盲目搜索

1.2 启发式搜索

1.3 博弈树搜索

1.4 高级搜索

2. 实验任务

2.1 盲目搜索解决迷宫问题（无需提交）

2.2 启发式搜索解决15-Puzzle问题

2.3 博弈树搜索实现象棋AI （无需提交）

2.4 遗传算法求解 TSP 问题

3. 作业提交说明

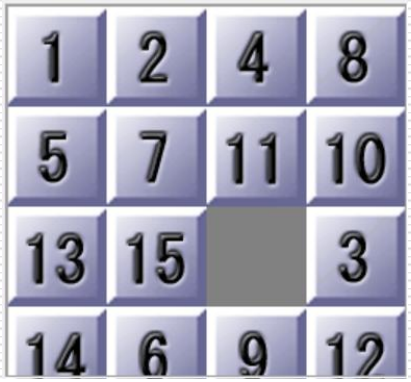
2.1 盲目搜索解决迷宫问题（无需提交）

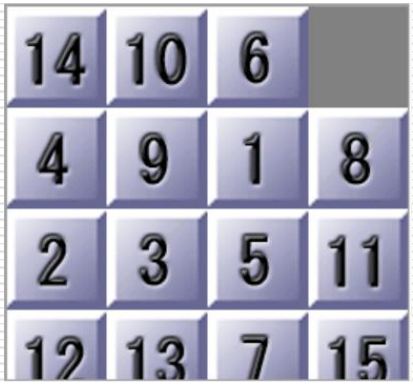
- 尝试利用DFS、BFS、深度受限算法、迭代加深算法、双向搜索算法解决迷宫问题:
  - S表示起点;
  - E表示终点;
  - 1表示墙;
  - 0是可通行。

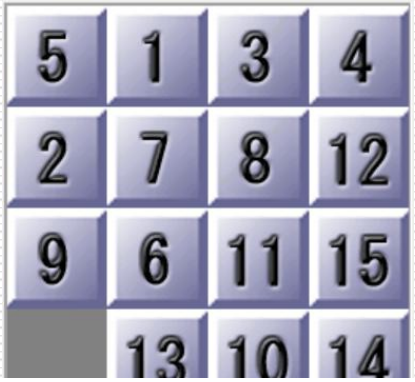
```
111111111111111111111111111111111111
100000000000000000000000000000000S1
101111111111111111111111101111111101
101100010001000000111111100011000001
101101010101011110111111111011011111
101101010101000000000000011011000001
101101010101010111100111000011111101
101001010100010000110111111110000001
101101010111111110110000000011011111
101101000110000000111111111011000001
100001111110111111100000011011111101
111111000000100000001111011010000001
100000011111101111101000011011011111
101111110000001000000011111011000001
100000000111111011111111111011001101
111111111100000000000000000011111101
1E000000000111111111111111000000001
111111111111111111111111111111111111
```

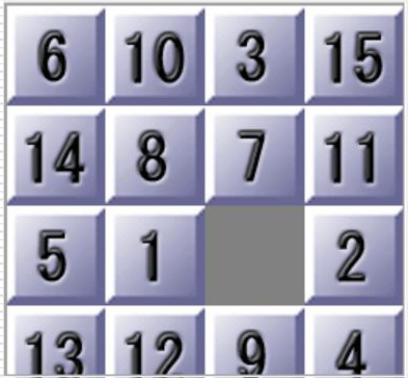
2.2 启发式搜索解决15-Puzzle问题

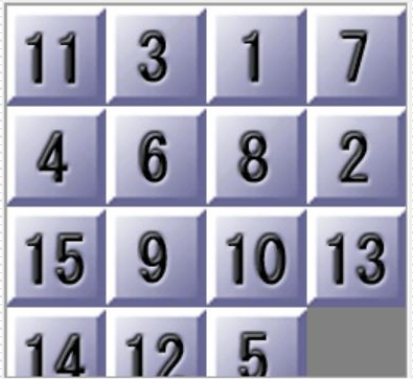
- 尝试使用A*与IDA*算法解决15-Puzzle问题，启发式函数可以自己选取，最好多尝试几种不同的启发式函数

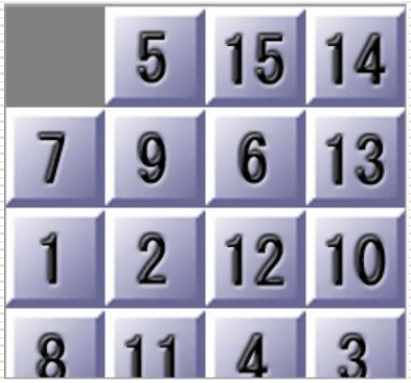








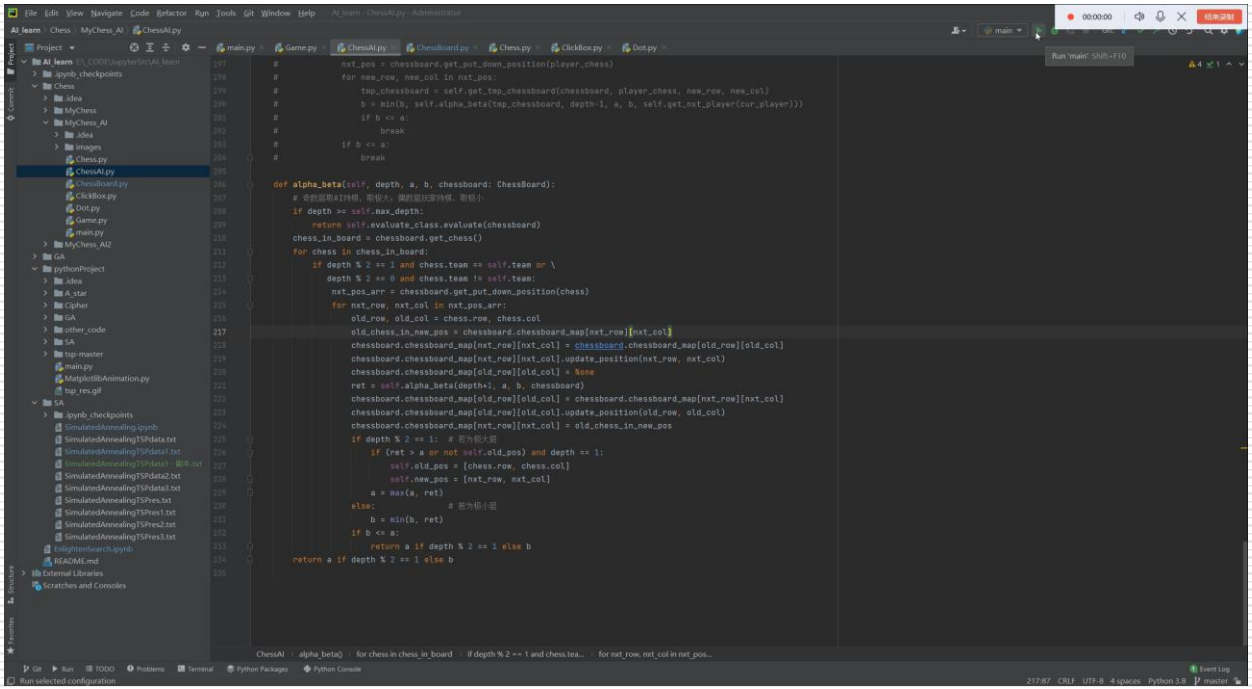




2.3 博弈树搜索实现象棋AI（无需提交）

- 编写一个中国象棋博弈程序，要求用alpha-beta剪枝算法，可以实现两个AI对弈。
  - 一方由人类点击或者AI算法控制。
  - 一方由内置规则AI控制。
  - 算法支持红黑双方互换

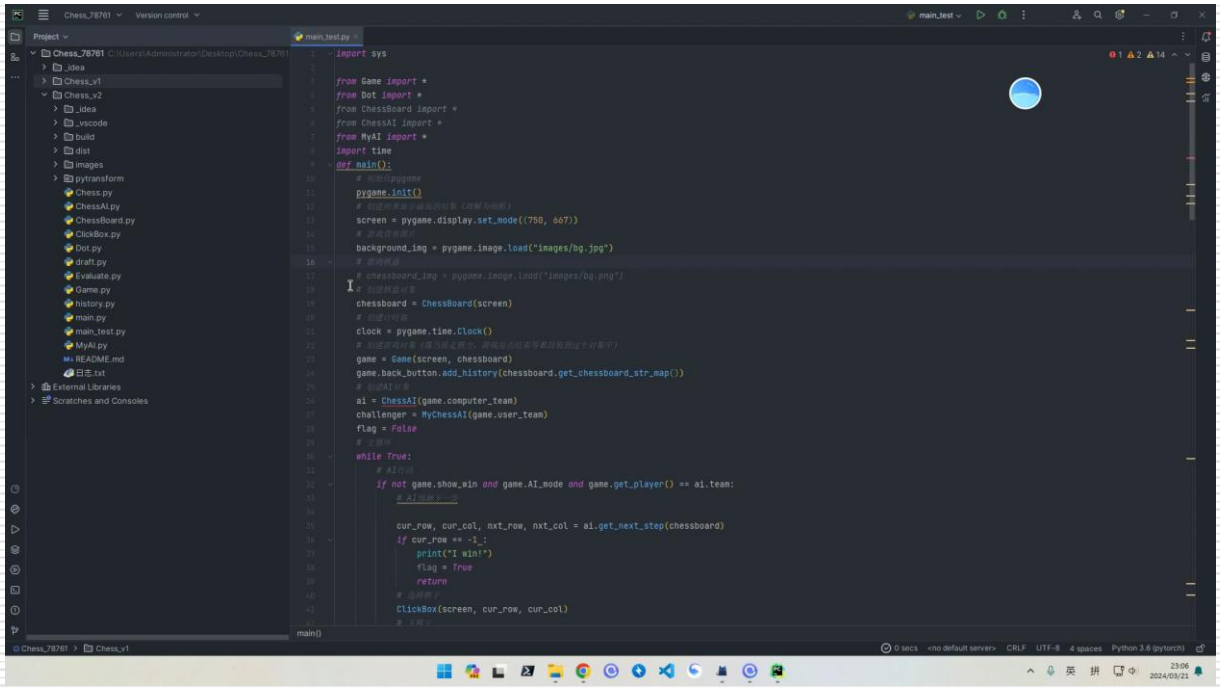
人类
Vs
内置规则



2.3 博弈树搜索实现象棋AI（无需提交）

- 编写一个中国象棋博弈程序，要求用alpha-beta剪枝算法，可以实现两个AI对弈。
  - 一方由人类点击或者AI算法控制。
  - 一方由内置规则AI控制。
  - 算法支持红黑双方互换

AI算法
Vs
内置规则



2.4 遗传算法求解 TSP 问题

利用遗传算法求解 TSP 问题

- 在National Traveling Salesman Problems (uwaterloo.ca)（https://www.math.uwaterloo.ca/tsp/world/countries.html）中任选两个TSP问题的数据集。
- 建议：
  - 对于规模较大的TSP问题, 遗传算法可能需要运行几分钟甚至几个小时的时间才能得到一个比较好的结果. 因此建议先用城市数较小的数据集测试算法正确与否, 再用城市数较大的数据集来评估算法性能。
  - 由于遗传算法是基于随机搜索的算法, 只运行一次算法的结果并不能反映算法的性能. 为了更好地分析遗传算法的性能, 应该以不同的初始随机种子或用不同的参数(例如种群数量, 变异概率等)多次运行算法, 这些需要在实验报告中呈现.

目录

1. 理论课内容回顾

1.1 盲目搜索

1.2 启发式搜索

1.3 博弈树搜索

1.4 高级搜索

2. 实验任务

2.1 盲目搜索解决迷宫问题（无需提交）

2.2 启发式搜索解决15-Puzzle问题

2.3 博弈树搜索实现象棋AI （无需提交）

2.4 遗传算法求解 TSP 问题

3. 作业提交说明

3. 作业提交说明

- 压缩包命名为：“学号_姓名_作业编号”，例：20260402_张三_实验3。
- 每次作业文件下包含两部分：code文件夹和实验报告PDF文件。
  - code文件夹：存放实验代码；
  - PDF文件格式参考发的模板。
- 如果需要更新提交的版本，则在后面加_v2，_v3。如第一版是“学号_姓名_作业编号.zip”，第二版是“学号_姓名_作业编号_v2.zip”，依此类推。
- 截至日期：**2026年4月23日晚24点**。
- 提交邮箱：zhangyc8@mail2.sysu.edu.cn。